Vitis AI en la VCK190 – Resumen de comandos

Oct 23, 2026 · @Mili

El modelo MNIST del tutorial 09-mnist_pyt corre en la DPU de la VCK190, compilado con Vitis AI 2.0 para la DPU DPUCVDX8G_ISA2_C32B3.

Dónde se ejecuta cada comando

Antes de pegar un comando, mirá el prompt de la terminal:

Prompt	Dónde estás
`diel@C-3531:~$`	Tu PC (Ubuntu 22.04)
`(vitis-ai-pytorch) Vitis-AI /workspace >`	Contenedor Docker de Vitis AI, en tu PC
`root@xilinx-vck190-2021_2:~#`	La placa VCK190

En el contenedor, `/workspace` es la carpeta `files` del tutorial. Lo que se genera ahí queda también en tu PC.

1. Instalar Docker y configurar el proxy (PC)

Docker se instala desde los repositorios de Ubuntu, porque `download.docker.com` está bloqueado en la red de INVAP.

```
sudo apt update
sudo apt install -y docker.io
sudo usermod -aG docker $USER   # luego cerrar sesión (o usar: newgrp docker)
docker --version
docker run hello-world
```

Proxy para que Docker descargue imágenes:

```
sudo mkdir -p /etc/systemd/system/docker.service.d
sudo tee /etc/systemd/system/docker.service.d/http-proxy.conf > /dev/null
<<EOF
[Service]
Environment="HTTP_PROXY=http://proxy.invap.com.ar:3128"
Environment="HTTPS_PROXY=http://proxy.invap.com.ar:3128"
Environment="NO_PROXY=localhost,127.0.0.1,10.0.0.0/8,.invap.com.ar,invap.com.
ar"
EOF
sudo systemctl daemon-reload
sudo systemctl restart docker
sudo systemctl show --property=Environment docker   # verificar (salir con q)
```

Proxy dentro de los contenedores (para `pip` o descargar MNIST):

```
mkdir -p ~/.docker
cat > ~/.docker/config.json <<EOF
{
  "proxies": {
    "default": {
      "httpProxy": "http://proxy.invap.com.ar:3128",
      "httpsProxy": "http://proxy.invap.com.ar:3128",
      "noProxy": "localhost,127.0.0.1,.invap.com.ar,invap.com.ar"
    }
  }
}
EOF
```

2. Descargar el tutorial (PC)

Se descarga solo la carpeta `09-mnist_pyt` de la rama 1.4. Los scripts sirven igual con el Docker 2.0.

```
git clone --depth 1 --filter=blob:none --sparse -b 1.4
https://github.com/Xilinx/Vitis-AI-Tutorials.git
cd Vitis-AI-Tutorials
git sparse-checkout set Design_Tutorials/09-mnist_pyt
cd Design_Tutorials/09-mnist_pyt/files
chmod +x *.sh   # sin esto, docker_run.sh da "Permiso denegado"
```

3. Preparar la placa

La placa arranca con la imagen de Vitis AI 2.0 (herramientas 2021.2). La imagen 1.4 (2020.2) no arranca: es para chips ES1, y esta VCK190 es de producción.

Hardware (con la placa apagada):

1. Grabar la imagen de SD de Vitis AI 2.0 para VCK190 con balenaEtcher.
2. SW1 (Versal) en ON, OFF, OFF, OFF → arranque desde SD.
3. SW11 (controlador del sistema) en ON, OFF, OFF, OFF.
4. Jumper J326 7-8 quitado. Si está puesto, no hay consola por USB.
5. La microSD del SYSCTRL en su ranura, y la tuya en la del Versal.

Consola UART (PC): abrirla antes de encender la placa. Parámetros: 115200, 8 bits, sin paridad, 1 bit de parada.

```
ls /dev/ttyUSB*
sudo screen /dev/ttyUSB1 115200   # ttyUSB1 = Versal UART0. Salir: Ctrl+A, K, Y
# login: root / root
```

En la placa: verificar la DPU y asignar la IP. La IP se pierde al reiniciar.

```
xdputil query          # DPU: DPUCVDX8G_ISA2_C32B3, Vitis AI 2.0.0
ifconfig eth0 192.168.1.10 netmask 255.255.255.0 up
ping -c 3 192.168.1.1
```

Desde la PC, entrar por red:

```
ssh-keygen -R 192.168.1.10
ssh root@192.168.1.10   # contraseña: root
```

4. Cuantizar y compilar con Vitis AI 2.0

El Docker tiene que ser de la misma versión que la placa (2.0), y la compilación tiene que decir `Target architecture: DPUCVDX8G_ISA2_C32B3`.

PC — lanzar el contenedor:

```
cd ~/Vitis-AI-Tutorials/Design_Tutorials/09-mnist_pyt/files
./docker_run.sh xilinx/vitis-ai-cpu:2.0.0.1103
```

Contenedor — entrenar (si hace falta) y cuantizar:

```
conda activate vitis-ai-pytorch
export BUILD=./build
export LOG=${BUILD}/logs
mkdir -p ${LOG}

python -u train.py -d ${BUILD} 2>&1 | tee ${LOG}/train.log   # solo si no hay
modelo entrenado
python -u quantize.py -d ${BUILD} --quant_mode calib 2>&1 | tee
${LOG}/quant_calib.log
python -u quantize.py -d ${BUILD} --quant_mode test  2>&1 | tee
${LOG}/quant_test.log
```

Contenedor — compilar con el fingerprint de la DPU de la placa y armar la carpeta:

```
echo '{"fingerprint":"0x602001036088231"}' > arch_vck190_isa2.json
vai_c_xir -x ${BUILD}/quant_model/CNN_int.xmodel -a arch_vck190_isa2.json -o
${BUILD}/compiled_model -n CNN_vck190
ls -l ${BUILD}/compiled_model/   # la fecha del .xmodel tiene que ser de
recién

rm -rf ${BUILD}/target_vck190
python -u target.py --target vck190 -d ${BUILD} 2>&1 | tee
${LOG}/target_vck190.log
exit
```

Las variables `BUILD` y `LOG` se pierden al salir del contenedor; hay que definirlas cada vez.

5. Copiar y ejecutar en la placa

La aplicación es `app_mt.py`; por defecto usa el modelo de la ZCU102, así que hay que pasarle el de la VCK190 con `-m`.

Placa — borrar la versión anterior:

```
rm -rf ~/target_vck190
```

PC (fuera del contenedor) — copiar la carpeta:

```
cd ~/Vitis-AI-Tutorials/Design_Tutorials/09-mnist_pyt/files
scp -r build/target_vck190 root@192.168.1.10:~/
```

Placa — ejecutar:

```
cd ~/target_vck190
python3 app_mt.py -h                                # ver opciones
python3 app_mt.py -m CNN_vck190.xmodel -t 1         # 1 hilo
python3 app_mt.py -m CNN_vck190.xmodel -t 4         # 4 hilos, más FPS
```

Si aparece `CHECK fingerprint fail`, el modelo se compiló para otra DPU: hay que recompilar (sección 4) y volver a copiar.

6. Resultados y datos clave

Dato	Valor
Precisión float32 (PC, Vitis AI 1.4)	98,78%
Precisión int8 (PC, Vitis AI 2.0)	98,81%
Precisión en la placa	Pendiente
FPS en la placa (1 / 4 hilos)	Pendiente
DPU de la placa	DPUCVDX8G_ISA2_C32B3
Fingerprint de la DPU	0x602001036088231
Docker para compilar	xilinx/vitis-ai-cpu:2.0.0.1103
Imagen de la placa	Vitis AI 2.0, PetaLinux 2021.2

Lecciones aprendidas

- Imagen de la placa, Docker y DPU tienen que ser de la misma versión. Si el fingerprint no coincide, la aplicación falla.
- Mirar el prompt antes de pegar cada comando: PC, contenedor o placa.
- La IP asignada con `ifconfig` se pierde al reiniciar la placa.
- Pegar en la terminal con Ctrl + Shift + V, no con Ctrl + V.
- Usar una microSD aparte para pruebas; no sobrescribir la de otro proyecto.